

Heterogeneous Database Migration to AWS PostgreSQL

Using AWS DMS & Schema Conversion Tool (SCT)

Oracle | SQL Server | MySQL/MariaDB -> PostgreSQL

Comprehensive Production Runbook with FAANG/MNC Standard Practices

Migration Methods | Step-by-Step Procedures | Data Type Mappings

Pre/Post Checklists | Cutover | Validation | Rollback | Terraform IaC

Version: 2.0 | Classification: Internal - Production Use

Table of Contents

PART I -- MIGRATION METHODS OVERVIEW

- 1.1 Oracle to PostgreSQL -- All Migration Methods
- 1.2 SQL Server to PostgreSQL -- All Migration Methods
- 1.3 MySQL/MariaDB to PostgreSQL -- All Migration Methods
- 1.4 FAANG/MNC Standard Practice Summary

PART II -- ORACLE TO POSTGRESQL RUNBOOK

- 2.1 Pre-Migration Assessment & Planning
- 2.2 AWS SCT -- Schema Conversion Procedure
- 2.3 Oracle Source Configuration for DMS
- 2.4 AWS DMS -- Replication Instance & Endpoints
- 2.5 AWS DMS -- Migration Task (Full Load + CDC)
- 2.6 Data Type Mapping -- Oracle to PostgreSQL
- 2.7 PL/SQL to PL/pgSQL Conversion Guide
- 2.8 Post-Migration Validation & Cutover

PART III -- SQL SERVER TO POSTGRESQL RUNBOOK

- 3.1 Pre-Migration Assessment & Planning
- 3.2 AWS SCT -- Schema Conversion for T-SQL
- 3.3 SQL Server Source Configuration for DMS
- 3.4 DMS Endpoints & Migration Task
- 3.5 Data Type Mapping -- SQL Server to PostgreSQL
- 3.6 T-SQL to PL/pgSQL Conversion Guide
- 3.7 Post-Migration Validation & Cutover

PART IV -- MYSQL/MARIADB TO POSTGRESQL RUNBOOK

- 4.1 Pre-Migration Assessment & Planning
- 4.2 AWS SCT -- Schema Conversion for MySQL
- 4.3 MySQL/MariaDB Source Configuration for DMS
- 4.4 DMS Endpoints & Migration Task
- 4.5 Alternative: pgloader (Single Command Migration)
- 4.6 Data Type Mapping -- MySQL/MariaDB to PostgreSQL
- 4.7 MySQL Dialect to PostgreSQL Conversion
- 4.8 Post-Migration Validation & Cutover

PART V -- CROSS-CUTTING CONCERNS (NEW)

- 5.1 AWS IAM Roles & Security Setup
- 5.2 Network Configuration (VPN / Direct Connect)
- 5.3 DMS CloudWatch Monitoring & Alerting
- 5.4 LOB Migration Strategies
- 5.5 Large Table Parallel Load Strategy
- 5.6 PostgreSQL Target Performance Tuning
- 5.7 Terraform IaC for DMS Infrastructure

PART VI -- APPENDICES

- A. AWS DMS Instance Sizing Guide
- B. Common DMS Error Codes & Resolution
- C. Rollback Strategy Template
- D. Migration Checklist (Printable)
- E. Cost Estimation Reference

PART I -- MIGRATION METHODS OVERVIEW

1.1 Oracle to PostgreSQL -- All Migration Methods

Seven primary approaches exist for migrating Oracle to PostgreSQL. The choice depends on database size, PL/SQL complexity, downtime tolerance, and whether the target is AWS-managed (RDS/Aurora) or self-managed.

#	Tool / Approach	Best For	Down-time	FAANG / MNC Usage
1	AWS DMS + SCT	Large enterprise DBs; CDC for minimal downtime	Min. (CDC)	Primary at Amazon, Netflix, Capital One
2	Ora2Pg (Open Source)	Full schema + PL/SQL conversion; cost assessment	Hours (batch)	Booking.com, GitLab, govt agencies
3	AWS DMS + Ora2Pg Hybrid	SCT for data, Ora2Pg for PL/SQL code	Min. (CDC)	Common FAANG pattern; best of both tools
4	pgloader + Ora2Pg	Non-AWS targets; self-managed PG	Hours (batch)	Zalando, smaller orgs
5	Oracle GoldenGate	Oracle-native CDC to flat files, then COPY	Min.	Legacy Oracle shops transitioning slowly
6	EDB Migration Toolkit	EDB Postgres Advanced Server targets	Hours	EDB enterprise customers
7	oracle_fdw (FDW)	Incremental/phased migration; coexistence	Zero (live)	Google-style phased migrations

INFO

FAANG Standard: Amazon and Netflix use AWS DMS + SCT as the primary path for Oracle-to-Aurora PostgreSQL. Schema conversion is done via SCT first, then PL/SQL packages that fail auto-conversion are manually rewritten using Ora2Pg's migration assessment report. Data migration uses Full Load + CDC mode.

1.2 SQL Server to PostgreSQL -- All Migration Methods

SQL Server migrations present unique challenges around T-SQL procedures, linked servers, SSIS packages, and SQL Agent jobs.

#	Tool / Approach	Best For	Down-time	FAANG / MNC Usage
1	AWS DMS + SCT	Enterprise migrations to RDS/Aurora PG	Min. (CDC)	Primary at Amazon, Stripe, Airbnb
2	Babelfish for Aurora PG	T-SQL app compat without full rewrite	Hours (setup)	Amazon internal; reduces T-SQL effort
3	pgloader	Direct MSSQL->PG data loading	Hours (batch)	Mid-size companies; fast for data-only
4	sqlserver2pgsql (Perl)	Schema conversion for self-managed PG	Hours	Open-source (Dalibo)

#	Tool / Approach	Best For	Down-time	FAANG / MNC Usage
5	Ora2Pg (MSSQL mode)	T-SQL schema + code conversion	Hours	Complementary to DMS for code conversion
6	SSMA + manual (hybrid)	SQL Server Migration Assistant + hand-tuning	Hours	Microsoft shops evaluating PG

INFO

FAANG Standard: Stripe and Airbnb use DMS + SCT with Babelfish as a transitional compatibility layer: (1) SCT converts DDL, (2) Babelfish handles T-SQL stored procs in Phase 1, (3) gradual rewrite to native PL/pgSQL in Phase 2, (4) DMS handles data migration with CDC throughout.

1.3 MySQL/MariaDB to PostgreSQL -- All Migration Methods

MySQL/MariaDB to PostgreSQL is a homogeneous-class migration (both open-source RDBMS) but still involves significant differences in SQL dialect, auto-increment handling, and character set management.

#	Tool / Approach	Best For	Down-time	FAANG / MNC Usage
1	AWS DMS + SCT	AWS-managed migrations; large datasets with CDC	Min. (CDC)	Primary at Amazon, Uber migrations
2	pgloader	Fast direct migration; single command	Hours (batch)	Widely used; created by PG core contributor
3	Ora2Pg (MySQL mode)	Full schema + stored proc conversion	Hours	Complementary to DMS
4	pg_chameleon	MySQL replica -> PG via logical replication	Min.	Novel approach; near-zero downtime
5	mysqldump + psql (manual)	Small databases; simple schemas	Hours	Dev/test environments
6	mysql_fdw (FDW)	Phased migration; coexistence period	Zero (live)	Google-style incremental migration

INFO

FAANG Standard: Uber's MySQL-to-PG migration used a phased approach: pgloader for initial schema + data, AWS DMS for ongoing CDC replication, and extensive dual-write validation before cutover. For simpler schemas, pgloader alone (single command) is sufficient.

1.4 FAANG/MNC Standard Practice Summary

Across major technology companies, migration follows a consistent five-phase framework:

- Assessment Phase (2-4 weeks): Run AWS SCT Assessment Report and/or Ora2Pg migration cost assessment. Categorize objects into auto-convertible vs. manual-rewrite buckets.
- Schema Conversion Phase (2-8 weeks): AWS SCT for DDL. Supplement with Ora2Pg for PL/SQL. Manual rewrite for complex packages, materialized views with refresh logic, and database links.

-
- Data Migration Phase (1-2 weeks): DMS Full Load + CDC. Parallel load with table segmentation for large tables (100M+ rows). C5 replication instances for heterogeneous migrations.
 - Application Conversion Phase (4-12 weeks): Convert ORM queries, embedded SQL, connection strings. Dual-write validation. Performance benchmarking.
 - Cutover Phase (1 day): Stop writes -> drain CDC lag -> switch DNS/connection strings -> validate -> go live. Rollback window: 2-4 hours with source DB preserved throughout.

PART II -- ORACLE TO POSTGRESQL DETAILED RUNBOOK

2.1 Pre-Migration Assessment & Planning

2.1.1 Source Database Discovery

Before any migration, perform a thorough inventory of the Oracle source database. Execute these queries to understand migration scope:

Oracle Source Discovery Queries

```
-- 1. Database Size Summary
SELECT tablespace_name,
       ROUND(SUM(bytes)/1024/1024/1024, 2) AS size_gb
FROM   dba_segments
GROUP BY tablespace_name
ORDER BY size_gb DESC;

-- 2. Schema Object Inventory
SELECT object_type, COUNT(*) AS obj_count
FROM   dba_objects
WHERE  owner = 'YOUR_SCHEMA'
      AND object_type IN ('TABLE', 'VIEW', 'INDEX', 'SEQUENCE',
                          'PROCEDURE', 'FUNCTION', 'PACKAGE', 'PACKAGE BODY',
                          'TRIGGER', 'MATERIALIZED VIEW', 'TYPE', 'SYNONYM')
GROUP BY object_type ORDER BY obj_count DESC;

-- 3. Table Row Counts (Top 20 largest)
SELECT table_name, num_rows,
       ROUND(num_rows * avg_row_len / 1024/1024/1024, 2) AS est_gb
FROM   dba_tables WHERE owner = 'YOUR_SCHEMA'
ORDER BY num_rows DESC NULLS LAST FETCH FIRST 20 ROWS ONLY;

-- 4. LOB Columns (special handling needed for DMS)
SELECT table_name, column_name, data_type
FROM   dba_tab_columns WHERE owner = 'YOUR_SCHEMA'
      AND data_type IN ('BLOB', 'CLOB', 'NCLOB', 'BFILE');

-- 5. Partitioned Tables
SELECT table_name, partitioning_type, partition_count
FROM   dba_part_tables WHERE owner = 'YOUR_SCHEMA';

-- 6. Foreign Key Dependencies (migration order)
SELECT a.table_name AS child, c_pk.table_name AS parent,
       a.constraint_name
FROM   dba_constraints a
JOIN   dba_constraints c_pk ON a.r_constraint_name = c_pk.constraint_name
WHERE  a.owner = 'YOUR_SCHEMA' AND a.constraint_type = 'R';
```

2.1.2 Ora2Pg Migration Cost Assessment

Run Ora2Pg's built-in assessment to get a difficulty score and estimated man-days. This is the industry-standard way to estimate effort before committing resources.

Ora2Pg Migration Assessment

```
# Install Ora2Pg (AlmaLinux 8/9)
sudo dnf install -y perl perl-DBI perl-DBD-Oracle perl-ExtUtils-MakeMaker
wget https://github.com/darold/ora2pg/archive/refs/tags/v24.3.tar.gz
tar xzf v24.3.tar.gz && cd ora2pg-24.3
perl Makefile.PL && make && sudo make install

# Generate migration assessment report
ora2pg -t SHOW_REPORT --estimate_cost \
  --source 'dbi:Oracle:host=oracle-prod;sid=ORCL;port=1521' \
  --user DMS_USER --password 'SecurePass123' \
  --schema YOUR_SCHEMA

# Output: object counts, difficulty level (A-E), estimated man-days,
#         total cost, specific conversion issues identified
```

NOTE

Ora2Pg Difficulty Levels: A = trivial (<1 day) | B = easy (1-5 days) | C = moderate (5-15 days, some PL/SQL rewrite) | D = hard (15-30 days, complex packages) | E = very hard (30+ days, extensive Oracle-specific features). Plan team allocation accordingly.

2.1.3 AWS SCT Assessment Report

In parallel, run AWS SCT assessment for AWS-specific conversion insights:

AWS SCT Assessment Setup

```
# Step 1: Download AWS SCT from AWS documentation site
# Step 2: Install JDBC drivers (ojdbc8.jar + postgresql-42.x.x.jar)
# Place in: /opt/sct/drivers/
# Step 3: Create New Project -> Source: Oracle, Target: Aurora PostgreSQL
# Step 4: Connect to Source Oracle
# Host: oracle-prod.company.internal Port: 1521 SID: ORCL
# User: DMS_USER (needs SELECT_CATALOG_ROLE)
# Step 5: Right-click schema -> Create Report
# Review: Executive Summary, Action Items, Object Conversion %
# Step 6: Export assessment as PDF for stakeholder review
```

2.2 AWS SCT -- Schema Conversion Procedure

After assessment, perform the actual schema conversion:

- In AWS SCT, right-click Oracle schema -> Convert Schema
- Blue icon = auto-converted | Red = requires manual | Orange = review needed
- Right-click target schema -> Apply to Database (pushes DDL to target PG)
- Or: Right-click -> Save as SQL to export DDL scripts for manual review

CRITICAL

CRITICAL -- Conversion Order: Always apply in dependency order: (1) CREATE EXTENSION (uuid-osp, pgcrypto), (2) CREATE TYPE, (3) CREATE TABLE, (4) CREATE INDEX, (5) CREATE FUNCTION/PROCEDURE, (6) CREATE TRIGGER, (7) CREATE VIEW/MATERIALIZED VIEW. Foreign keys should be applied AFTER DMS full load completes.

2.2.1 Manual Conversion Items (Common Oracle -> PG Issues)

Oracle Feature	PostgreSQL Equivalent	Notes
CONNECT BY (hierarchical)	WITH RECURSIVE CTE	Rewrite all hierarchical queries
ROWNUM	ROW_NUMBER() OVER() or LIMIT	Replace with window functions
SYSDATE	CURRENT_TIMESTAMP or NOW()	NOW() is transaction-scoped
NVL()	COALESCE()	Direct replacement (ANSI SQL)
DECODE()	CASE WHEN ... THEN ... END	Standard CASE expressions
DBMS_OUTPUT.PUT_LINE	RAISE NOTICE	Replace in all PL/pgSQL functions
Oracle Packages	Schema + Functions	Flatten packages into standalone functions
DBMS_LOB	PG large object functions	lo_import, lo_export, lo_read, lo_write
Global Temp Tables	TEMP TABLE (session/tx)	CREATE TEMP TABLE or UNLOGGED TABLE
Mat. View REFRESH	REFRESH MAT. VIEW	No FAST REFRESH; use CONCURRENTLY
Database Links	postgres_fdw / dblink	Use Foreign Data Wrappers
Synonyms	SET search_path / Views	Public synonyms -> search_path
Oracle Partitioning	PG Declarative Partitioning	RANGE/LIST/HASH from PG 10+
Bitmap Indexes	Standard B-tree / GIN	GIN for low-cardinality columns
Function-based Indexes	Expression Indexes	CREATE INDEX ON table (expression)
SEQUENCES CACHE/NOCACHE	CREATE SEQUENCE CACHE n	Match Oracle CACHE value

2.3 Oracle Source Configuration for AWS DMS

Configure the Oracle source for DMS change data capture: ARCHIVELOG mode, supplemental logging, and appropriate grants.

Step 1: Enable ARCHIVELOG Mode (requires restart)

Enable ARCHIVELOG Mode

```
-- Check current mode
SELECT log_mode FROM v$database;

-- If NOARCHIVELOG, enable it:
SHUTDOWN IMMEDIATE;
STARTUP MOUNT;
ALTER DATABASE ARCHIVELOG;
ALTER DATABASE OPEN;

-- Verify: should show ARCHIVELOG
SELECT log_mode FROM v$database;
```

Step 2: Enable Supplemental Logging

Enable Supplemental Logging

```
-- Database-level (minimum requirement)
ALTER DATABASE ADD SUPPLEMENTAL LOG DATA;

-- Primary key logging for all tables (recommended)
ALTER DATABASE ADD SUPPLEMENTAL LOG DATA (PRIMARY KEY) COLUMNS;

-- For tables WITHOUT primary keys, add ALL columns logging:
ALTER TABLE your_schema.no_pk_table
  ADD SUPPLEMENTAL LOG DATA (ALL) COLUMNS;

-- Verify
SELECT supplemental_log_data_min, supplemental_log_data_pk
FROM v$database;
```

Step 3: Create DMS User with Required Grants

Create DMS User & Grants (Oracle Source)

```
CREATE USER dms_user IDENTIFIED BY 'ComplexP@ss2024!'
  DEFAULT TABLESPACE users TEMPORARY TABLESPACE temp
  QUOTA UNLIMITED ON users;

-- Core privileges
GRANT CREATE SESSION          TO dms_user;
GRANT SELECT ANY TABLE      TO dms_user;
GRANT SELECT ANY TRANSACTION TO dms_user;
GRANT SELECT ON V_$ARCHIVED_LOG TO dms_user;
GRANT SELECT ON V_$LOG        TO dms_user;
GRANT SELECT ON V_$LOGFILE     TO dms_user;
GRANT SELECT ON V_$LOGMNR_LOGS TO dms_user;
GRANT SELECT ON V_$LOGMNR_CONTENTS TO dms_user;
GRANT SELECT ON V_$DATABASE    TO dms_user;
GRANT SELECT ON V_$THREAD      TO dms_user;
GRANT SELECT ON V_$PARAMETER    TO dms_user;
GRANT SELECT ON V_$NLS_PARAMETERS TO dms_user;
GRANT SELECT ON V_$TIMEZONE_NAMES TO dms_user;
GRANT SELECT ON V_$TRANSACTION TO dms_user;

-- LogMiner privileges (required for CDC)
GRANT EXECUTE ON DBMS_LOGMNR TO dms_user;
GRANT LOGMINING TO dms_user; -- Oracle 12c+

-- For 12c+ CDB/PDB architecture:
ALTER SESSION SET CONTAINER = CDB$ROOT;
GRANT SET CONTAINER TO dms_user CONTAINER = ALL;
```

WARNING

RDS for Oracle: ARCHIVELOG is enabled by default. For supplemental logging use:

```
exec rdsadmin.rdsadmin_util.alter_supplemental_logging('ADD');
exec rdsadmin.rdsadmin_util.alter_supplemental_logging('ADD','PRIMARY KEY');
```

2.4 AWS DMS -- Replication Instance & Endpoints

2.4.1 Create Replication Instance

Create Replication Instance (AWS CLI)

```
aws dms create-replication-instance \  
  --replication-instance-identifier oracle-to-pg-ri \  
  --replication-instance-class dms.c5.2xlarge \  
  --allocated-storage 200 \  
  --vpc-security-group-ids sg-0abc123def456 \  
  --replication-subnet-group-identifier dms-subnet-group \  
  --multi-az \  
  --engine-version 3.5.3 \  
  --publicly-accessible false \  
  --tags Key=Project,Value=OracleToPG \  
        Key=Environment,Value=Production  
  
# Wait for instance to become available  
aws dms wait replication-instance-available \  
  --filters Name=replication-instance-id,Values=oracle-to-pg-ri
```

TIP

Instance Sizing: dms.c5.large = up to 100 GB, <50 tables | dms.c5.xlarge = 100-500 GB, 50-200 tables | dms.c5.2xlarge = 500 GB-2 TB, 200+ tables (recommended for production Oracle) | dms.c5.4xlarge = 2+ TB, complex transformations. Always use C5 for heterogeneous migrations (CPU-intensive).

2.4.2 Create Source Endpoint (Oracle)

Create Oracle Source Endpoint

```
aws dms create-endpoint \  
  --endpoint-identifier oracle-source-prod \  
  --endpoint-type source \  
  --engine-name oracle \  
  --server-name oracle-prod.company.internal \  
  --port 1521 \  
  --database-name ORCL \  
  --username dms_user \  
  --password 'ComplexP@ss2024!' \  
  --extra-connection-attributes \  
    'useLogMinerReader=Y;useBfile=Y;addSupplementalLogging=Y;\  
    retryInterval=120;archivedLogDestId=1;\br/>    numberDatatypeScale=-2;archivedLogsOnly=N'  
  
# For Oracle ASM: add 'asmServer=+ASM;asmUser=ASMSNMP;asmPassword=xxx'  
# For RDS Oracle: add 'useLogMinerReader=Y;useBfile=Y'
```

2.4.3 Create Target Endpoint (PostgreSQL)

Create PostgreSQL Target Endpoint

```
aws dms create-endpoint \  
  --endpoint-identifier pg-target-aurora \  
  --endpoint-type target \  
  --engine-name aurora-postgresql \  
  --server-name mydb.cluster-abc123.us-east-1.rds.amazonaws.com \  
  --port 5432 \  
  --database-name myapp_db \  
  --username dms_admin \  
  --password 'TargetP@ss2024!' \  
  --extra-connection-attributes \  
    'maxFileSize=1048576;executeTimeout=3600'  
  
# For self-managed PostgreSQL (RDS or EC2): --engine-name postgres
```

2.4.4 Test Endpoint Connections

Test Endpoint Connections

```
# Test Source  
aws dms test-connection \  
  --replication-instance-arn arn:aws:dms:us-east-1:123456:rep:XXXX \  
  --endpoint-arn arn:aws:dms:us-east-1:123456:endpoint:oracle-source  
  
# Test Target  
aws dms test-connection \  
  --replication-instance-arn arn:aws:dms:us-east-1:123456:rep:XXXX \  
  --endpoint-arn arn:aws:dms:us-east-1:123456:endpoint:pg-target  
  
# Verify: both should show 'successful' status
```

2.5 AWS DMS -- Migration Task (Full Load + CDC)

For production Oracle migrations, use Full Load + CDC to minimize downtime.

2.5.1 Table Mapping Rules (JSON)

Table Mapping Rules -- table-mappings.json

```
{
  "rules": [
    {
      "rule-type": "selection",
      "rule-id": "1",
      "rule-name": "select-all-tables",
      "object-locator": {
        "schema-name": "YOUR_SCHEMA",
        "table-name": "%"
      },
      "rule-action": "include"
    },
    {
      "rule-type": "transformation",
      "rule-id": "2",
      "rule-name": "convert-schema-lowercase",
      "rule-action": "convert-lowercase",
      "rule-target": "schema",
      "object-locator": { "schema-name": "YOUR_SCHEMA" }
    },
    {
      "rule-type": "transformation",
      "rule-id": "3",
      "rule-name": "convert-table-lowercase",
      "rule-action": "convert-lowercase",
      "rule-target": "table",
      "object-locator": {
        "schema-name": "YOUR_SCHEMA", "table-name": "%"
      }
    },
    {
      "rule-type": "transformation",
      "rule-id": "4",
      "rule-name": "convert-column-lowercase",
      "rule-action": "convert-lowercase",
      "rule-target": "column",
      "object-locator": {
        "schema-name": "YOUR_SCHEMA",
        "table-name": "%", "column-name": "%"
      }
    }
  ]
}
```

CRITICAL

CRITICAL -- Lowercase: Oracle stores names in UPPERCASE; PostgreSQL in lowercase. You MUST include convert-lowercase rules for schema, table, AND column names, or queries will fail with 'relation does not exist' errors.

2.5.2 Task Settings (Key Parameters)

Task Settings -- task-settings.json

```
{
  "TargetMetadata": {
    "SupportLobs": true,
    "LimitedSizeLobMode": true,
    "LobMaxSize": 32768,
    "BatchApplyEnabled": true
  },
  "FullLoadSettings": {
    "TargetTablePrepMode": "TRUNCATE_BEFORE_LOAD",
    "CreatePkAfterFullLoad": true,
    "StopTaskCachedChangesApplied": true,
    "MaxFullLoadSubTasks": 16,
    "TransactionConsistencyTimeout": 600,
    "CommitRate": 50000
  },
  "Logging": {
    "EnableLogging": true,
    "LogComponents": [
      {"Id": "SOURCE_UNLOAD", "Severity": "LOGGER_SEVERITY_DEFAULT"},
      {"Id": "TARGET_LOAD", "Severity": "LOGGER_SEVERITY_DEFAULT"},
      {"Id": "SOURCE_CAPTURE", "Severity": "LOGGER_SEVERITY_DEFAULT"},
      {"Id": "TARGET_APPLY", "Severity": "LOGGER_SEVERITY_INFO"}
    ]
  },
  "ValidationSettings": {
    "EnableValidation": true,
    "ThreadCount": 5,
    "ValidationMode": "ROW_LEVEL"
  }
}
```

2.5.3 Create and Start Migration Task

Create & Start Migration Task

```
# Create task
aws dms create-replication-task \
  --replication-task-identifier ora-to-pg-full-cdc \
  --replication-instance-arn arn:aws:dms:us-east-1:123456:rep:XXXX \
  --source-endpoint-arn arn:aws:dms:...:endpoint:oracle-source \
  --target-endpoint-arn arn:aws:dms:...:endpoint:pg-target \
  --migration-type full-load-and-cdc \
  --table-mappings file://table-mappings.json \
  --replication-task-settings file://task-settings.json

# Wait, then start
aws dms wait replication-task-ready \
  --filters Name=replication-task-id,Values=ora-to-pg-full-cdc
aws dms start-replication-task \
  --replication-task-arn arn:aws:dms:...:task:XXXX \
  --start-replication-task-type start-replication

# Monitor progress
aws dms describe-replication-tasks \
  --filters Name=replication-task-id,Values=ora-to-pg-full-cdc \
  --query 'ReplicationTasks[0].ReplicationTaskStats'
```

2.6 Data Type Mapping -- Oracle to PostgreSQL

Oracle Data Type	PostgreSQL Equivalent	Notes
NUMBER(p,s)	NUMERIC(p,s)	Exact match; arbitrary precision
NUMBER (no precision)	DOUBLE PRECISION	Or NUMERIC if exact precision needed
NUMBER(1-4,0)	SMALLINT	Optimization for small integers
NUMBER(5-9,0)	INTEGER	Standard 32-bit integer
NUMBER(10-18,0)	BIGINT	64-bit integer
FLOAT / BINARY_DOUBLE	DOUBLE PRECISION	IEEE 754 double
BINARY_FLOAT	REAL	32-bit floating point
VARCHAR2(n) / NVARCHAR2(n)	VARCHAR(n)	PG is UTF-8 native
CHAR(n)	CHAR(n)	Consider VARCHAR for PG
CLOB / NCLOB	TEXT	Unlimited length in PG
BLOB	BYTEA	Or Large Objects for > 1GB
RAW(n) / LONG RAW	BYTEA	Binary data
DATE	TIMESTAMP(0)	Oracle DATE includes time! #1 error
TIMESTAMP	TIMESTAMP	Direct mapping
TIMESTAMP WITH TZ	TIMESTAMPTZ	Direct mapping
INTERVAL YEAR TO MONTH	INTERVAL	PG INTERVAL is more flexible
INTERVAL DAY TO SECOND	INTERVAL	Direct mapping
XMLTYPE	XML	Native XML type in PG
SDO_GEOMETRY	GEOMETRY (PostGIS)	Requires PostGIS extension
ROWID	No direct equivalent	Use CTID or synthetic PK
BOOLEAN (PL/SQL only)	BOOLEAN	Native TRUE/FALSE/NULL

CRITICAL

CRITICAL -- Oracle DATE: Oracle DATE stores BOTH date AND time (to the second). PostgreSQL DATE is date-only. You MUST map Oracle DATE -> PostgreSQL TIMESTAMP(0), NOT DATE. This is the #1 most common mapping error.

WARNING

NULL Character (U+0000): Oracle allows NULL chars in VARCHAR2/CLOB. PostgreSQL does NOT. DMS auto-converts U+0000 to space (U+0020). Verify this doesn't affect application logic.

2.7 PL/SQL to PL/pgSQL Conversion Guide

Converting PL/SQL is the most labor-intensive part. Key examples:

Oracle PL/SQL -- Before Conversion

```
-----  
-- ORACLE PL/SQL (Before)  
-----  
  
CREATE OR REPLACE PROCEDURE update_salary(  
    p_emp_id    IN  NUMBER,  
    p_pct       IN  NUMBER,  
    p_new_sal   OUT NUMBER  
) AS  
    v_current_sal employees.salary%TYPE;  
BEGIN  
    SELECT salary INTO v_current_sal  
    FROM   employees WHERE emp_id = p_emp_id;  
    p_new_sal := v_current_sal * (1 + p_pct / 100);  
    UPDATE employees SET salary = p_new_sal  
    WHERE  emp_id = p_emp_id;  
    DBMS_OUTPUT.PUT_LINE('Salary updated to: ' || p_new_sal);  
    COMMIT;  
EXCEPTION  
    WHEN NO_DATA_FOUND THEN  
        DBMS_OUTPUT.PUT_LINE('Employee not found');  
        RAISE;  
END update_salary;  
/
```

PostgreSQL PL/pgSQL -- After Conversion

```
-----  
-- POSTGRES PL/pgSQL (After)  
-----  
  
CREATE OR REPLACE PROCEDURE update_salary(  
    p_emp_id    IN  BIGINT,  
    p_pct       IN  NUMERIC,  
    INOUT p_new_sal NUMERIC DEFAULT 0  
) LANGUAGE plpgsql AS $$  
DECLARE  
    v_current_sal employees.salary%TYPE;  
BEGIN  
    SELECT salary INTO STRICT v_current_sal  
    FROM   employees WHERE emp_id = p_emp_id;  
    p_new_sal := v_current_sal * (1 + p_pct / 100);  
    UPDATE employees SET salary = p_new_sal  
    WHERE  emp_id = p_emp_id;  
    RAISE NOTICE 'Salary updated to: %', p_new_sal;  
    -- No COMMIT in PG procedures (auto-commit or caller TX)  
EXCEPTION  
    WHEN NO_DATA_FOUND THEN  
        RAISE NOTICE 'Employee not found: %', p_emp_id;  
        RAISE;  
END;  
$$;  
-- Call: CALL update_salary(101, 10, NULL);
```

Key PL/SQL -> PL/pgSQL Conversion Rules:

- Replace AS/IS block with LANGUAGE plpgsql AS \$\$... \$\$;
- OUT parameters become INOUT (PG procedures lack true OUT params)
- Add STRICT to SELECT INTO to raise NO_DATA_FOUND

- Replace DBMS_OUTPUT.PUT_LINE with RAISE NOTICE '%', value
- Remove explicit COMMIT/ROLLBACK (PG auto-commit or caller manages TX)
- Replace NUMBER -> NUMERIC/BIGINT, VARCHAR2 -> VARCHAR/TEXT
- Replace NVL() -> COALESCE(), SYSDATE -> CURRENT_TIMESTAMP
- Replace DECODE() -> CASE WHEN ... END
- Flatten Oracle Packages into schema-qualified standalone functions

2.8 Post-Migration Validation & Cutover

2.8.1 Data Validation Queries

Data Validation -- Row Counts

```
-- Run on BOTH source (Oracle) and target (PostgreSQL)

-- Oracle: row counts
SELECT table_name, num_rows FROM dba_tables
WHERE owner = 'YOUR_SCHEMA' ORDER BY table_name;

-- PostgreSQL: row counts
SELECT schemaname || '.' || relname AS table_name,
       n_live_tup AS row_count
FROM   pg_stat_user_tables
WHERE  schemaname = 'your_schema' ORDER BY relname;

-- Spot-check critical tables with actual COUNT(*):
-- Oracle:      SELECT COUNT(*) FROM your_schema.orders;
-- PostgreSQL: SELECT COUNT(*) FROM your_schema.orders;
```

2.8.2 Sequence Reset (CRITICAL)

Reset Sequences After Migration

```
-- DMS does NOT migrate sequence values!
-- After full load, reset all sequences to MAX(id) + 1.
DO $$
DECLARE r RECORD; max_val BIGINT;
BEGIN
  FOR r IN SELECT sequencename AS seq_name,
                 REPLACE(sequencename, '_seq', '') AS table_hint
            FROM pg_sequences WHERE schemaname = 'your_schema'
  LOOP
    EXECUTE format('SELECT COALESCE(MAX(id),0) FROM your_schema.%I',
                  r.table_hint) INTO max_val;
    EXECUTE format('SELECT setval(''your_schema.%I'', %s)',
                  r.seq_name, max_val + 1);
    RAISE NOTICE 'Reset % to %', r.seq_name, max_val + 1;
  END LOOP;
END; $$;
```

2.8.3 Production Cutover Procedure

CRITICAL

Cutover Checklist (Oracle -> PostgreSQL):

1. Announce maintenance window (schedule 2-4 hours)
2. Stop all application writes to Oracle source
3. Wait for DMS CDC lag = 0 (monitor CDCLatencyTarget)
4. Stop the DMS replication task
5. Run validation queries (row counts + sample checksums)
6. Reset all sequences to MAX(id) + 1
7. Create secondary indexes deferred during load
8. Enable foreign key constraints
9. Run ANALYZE on all tables
10. Update application connection strings
11. Deploy PG-compatible application code
12. Smoke test critical workflows
13. Monitor error logs for 30 minutes
14. ROLLBACK? Revert connection strings (source preserved)
15. SUCCESS? Schedule Oracle decommission (keep 30 days)

PART III -- SQL SERVER TO POSTGRESQL RUNBOOK

3.1 Pre-Migration Assessment & Planning

3.1.1 Source Database Discovery

SQL Server Source Discovery

```
-- Database Size
SELECT DB_NAME() AS database_name,
       CAST(SUM(size * 8.0/1024/1024) AS DECIMAL(10,2)) AS size_gb
FROM sys.database_files;

-- Object Inventory
SELECT type_desc, COUNT(*) AS obj_count
FROM sys.objects WHERE is_ms_shipped = 0
GROUP BY type_desc ORDER BY obj_count DESC;

-- Table Sizes (Top 20)
SELECT TOP 20 s.name + '.' + t.name AS table_name,
             SUM(p.rows) AS row_count,
             CAST(SUM(a.total_pages)*8.0/1024/1024 AS DECIMAL(10,2)) AS size_gb
FROM sys.tables t JOIN sys.schemas s ON t.schema_id=s.schema_id
JOIN sys.indexes i ON t.object_id=i.object_id
JOIN sys.partitions p ON i.object_id=p.object_id AND i.index_id=p.index_id
JOIN sys.allocation_units a ON p.partition_id=a.container_id
WHERE i.index_id IN (0,1)
GROUP BY s.name, t.name ORDER BY size_gb DESC;

-- T-SQL Stored Procedures & Functions count
SELECT type_desc, COUNT(*) FROM sys.objects
WHERE type IN ('P','FN','IF','TF','TR') AND is_ms_shipped=0
GROUP BY type_desc;

-- SQL Agent Jobs (manual migration to pg_cron)
SELECT j.name, j.enabled, js.step_name, js.subsystem
FROM msdb.dbo.sysjobs j
JOIN msdb.dbo.sysjobsteps js ON j.job_id=js.job_id
ORDER BY j.name, js.step_id;
```

3.2 AWS SCT -- Schema Conversion for T-SQL

SQL Server Feature	PostgreSQL Equivalent	Notes
IDENTITY(1,1)	GENERATED ALWAYS AS IDENTITY	Or SERIAL/BIGSERIAL (legacy)
GETDATE()	CURRENT_TIMESTAMP / NOW()	Direct replacement
ISNULL()	COALESCE()	ANSI SQL standard
TOP n	LIMIT n	Move from SELECT to end of query
WITH (NOLOCK)	Remove entirely	PG MVCC; no reader locks
@@IDENTITY	INSERT ... RETURNING id	Or CURRVAL()
#temp tables	CREATE TEMP TABLE	Syntax differs slightly

SQL Server Feature	PostgreSQL Equivalent	Notes
@table variables	CREATE TEMP TABLE	PG has no table variables
PRINT	RAISE NOTICE	Replace in all procedures
TRY...CATCH	BEGIN...EXCEPTION...END	PL/pgSQL exception handling
nvarchar(MAX)	TEXT	PG TEXT is unlimited
Computed Columns	Generated Columns (PG 12+)	GENERATED ALWAYS AS (expr) STORED
Filtered Indexes	Partial Indexes	CREATE INDEX ... WHERE condition
CROSS APPLY	LATERAL JOIN	Direct equivalent
MERGE statement	INSERT ON CONFLICT	Different syntax entirely
Schemas (dbo)	Schema (public)	Map dbo -> public or custom
RAISERROR	RAISE EXCEPTION	Different syntax
STRING_AGG (2017+)	STRING_AGG (PG 9.0+)	Same function name

3.3 SQL Server Source Configuration for DMS

SQL Server Source Configuration (Combined)

```
-- STEP 1: Enable MS-CDC on the database
USE YourDatabase;
EXEC sys.sp_cdc_enable_db;

-- Enable MS-CDC on each table
EXEC sys.sp_cdc_enable_table
    @source_schema = N'dbo', @source_name = N'orders',
    @role_name = NULL, @supports_net_changes = 1;

-- Generate CDC enable statements for ALL tables:
SELECT 'EXEC sys.sp_cdc_enable_table @source_schema=N''' +
    s.name + ''', @source_name=N''' + t.name +
    ''', @role_name=NULL, @supports_net_changes=1;'
FROM sys.tables t JOIN sys.schemas s ON t.schema_id=s.schema_id
WHERE t.is_ms_shipped = 0;

-- STEP 2: Ensure Full Recovery Model
ALTER DATABASE YourDatabase SET RECOVERY FULL;

-- STEP 3: Create DMS User
USE master; CREATE LOGIN dms_user WITH PASSWORD='ComplexP@ss2024!';
USE YourDatabase; CREATE USER dms_user FOR LOGIN dms_user;
GRANT VIEW DATABASE STATE TO dms_user;
GRANT VIEW DEFINITION TO dms_user;
GRANT SELECT ON SCHEMA::dbo TO dms_user;
EXEC sp_addrolemember 'db_datareader', 'dms_user';

-- For RDS SQL Server:
-- EXEC msdb.dbo.rds_cdc_enable_db 'YourDatabase';
```

WARNING

Without MS-CDC: If you cannot enable MS-CDC (e.g., Standard Edition), DMS can read the transaction log directly. Requires sysadmin role for dms_user and Full recovery model with regular log backups.

3.4 DMS Endpoints & Migration Task

SQL Server DMS Endpoint & Task

```
# SQL Server Source Endpoint
aws dms create-endpoint \
  --endpoint-identifier mssql-source-prod \
  --endpoint-type source \
  --engine-name sqlserver \
  --server-name mssql-prod.company.internal \
  --port 1433 --database-name YourDatabase \
  --username dms_user --password 'ComplexP@ss2024!' \
  --extra-connection-attributes \
    'safeguardPolicy=RELY_ON_SQL_SERVER_REPLICATION_AGENT;\
    useBcpFullLoad=Y'

# Table Mappings -- rename dbo -> public, lowercase tables
# Use same JSON structure as Oracle (Section 2.5.1)
# but with "schema-name": "dbo" and rename rule:
#   "rule-action": "rename", "value": "public"
```

3.5 Data Type Mapping -- SQL Server to PostgreSQL

SQL Server Type	PostgreSQL Type	Notes
INT / BIGINT / SMALLINT	INTEGER / BIGINT / SMALLINT	Direct mapping
TINYINT	SMALLINT	PG has no TINYINT
BIT	BOOLEAN	Map 0/1 to FALSE/TRUE
DECIMAL(p,s) / NUMERIC	NUMERIC(p,s)	Direct mapping
MONEY / SMALLMONEY	NUMERIC(19,4) / NUMERIC(10,4)	No MONEY type in PG
FLOAT / REAL	DOUBLE PRECISION / REAL	IEEE 754
CHAR(n) / VARCHAR(n)	CHAR(n) / VARCHAR(n)	Direct mapping
VARCHAR(MAX) / NVARCHAR(MAX)	TEXT	Unlimited in PG
NCHAR(n) / NVARCHAR(n)	CHAR(n) / VARCHAR(n)	PG is UTF-8 native
BINARY / VARBINARY / IMAGE	BYTEA	Binary data
DATE / TIME	DATE / TIME	Direct mapping
DATETIME	TIMESTAMP(3)	3ms precision like SQL Server
DATETIME2	TIMESTAMP	Up to 100ns SS vs 1us PG
DATETIMEOFFSET	TIMESTAMPTZ	Timezone-aware
SMALLDATETIME	TIMESTAMP(0)	Minute precision

SQL Server Type	PostgreSQL Type	Notes
UNIQUEIDENTIFIER	UUID	Requires uuid-osp extension
XML	XML	Direct mapping
GEOGRAPHY / GEOMETRY	PostGIS geometry	Requires PostGIS extension
HIERARCHYID	LTREE or custom	No direct equiv; use ltree
ROWVERSION / TIMESTAMP	No equivalent	Remove or trigger-based
SQL_VARIANT	TEXT or JSONB	Evaluate per use case

3.6 T-SQL to PL/pgSQL Conversion Guide

SQL Server T-SQL -- Before

```

-----
-- SQL SERVER T-SQL (Before)
-----

CREATE PROCEDURE dbo.usp_GetOrdersByDate
    @StartDate DATETIME, @EndDate DATETIME,
    @Status NVARCHAR(20) = NULL
AS BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        SELECT o.OrderID, o.OrderDate, c.CustomerName,
               ISNULL(o.ShipDate, '1900-01-01') AS ShipDate
        FROM dbo.Orders o
        INNER JOIN dbo.Customers c ON o.CustomerID = c.CustomerID
        WHERE o.OrderDate BETWEEN @StartDate AND @EndDate
              AND (@Status IS NULL OR o.Status = @Status)
        ORDER BY o.OrderDate DESC;
    END TRY
    BEGIN CATCH
        PRINT 'Error: ' + ERROR_MESSAGE();
        THROW;
    END CATCH
END;

```

```

-----
-- PostgreSQL PL/pgSQL (After)
-----
CREATE OR REPLACE FUNCTION get_orders_by_date(
    p_start TIMESTAMP, p_end TIMESTAMP,
    p_status VARCHAR DEFAULT NULL
) RETURNS TABLE (
    order_id BIGINT, order_date TIMESTAMP,
    customer_name VARCHAR, ship_date TIMESTAMP
) LANGUAGE plpgsql AS $$
BEGIN
    RETURN QUERY
    SELECT o.order_id, o.order_date, c.customer_name,
           COALESCE(o.ship_date, '1900-01-01'::TIMESTAMP)
    FROM orders o
    INNER JOIN customers c ON o.customer_id = c.customer_id
    WHERE o.order_date BETWEEN p_start AND p_end
           AND (p_status IS NULL OR o.status = p_status)
    ORDER BY o.order_date DESC;
EXCEPTION
    WHEN OTHERS THEN
        RAISE NOTICE 'Error: %', SQLERRM; RAISE;
END; $$;
-- Call: SELECT * FROM get_orders_by_date('2024-01-01', '2024-12-31');

```

3.7 Post-Migration Validation & Cutover

Follow Section 2.8 pattern. SQL Server-specific additions:

- Migrate SQL Agent Jobs -> pg_cron or OS-level cron
- Migrate SSIS packages -> AWS Glue or custom ETL scripts
- Migrate linked servers -> postgres_fdw
- Migrate SSRS reports -> AWS QuickSight or Grafana
- Convert .NET: SqlConnection -> NpgsqlConnection
- Replace SQL Server ODBC driver with psqLODBC

PART IV -- MYSQL/MARIADB TO POSTGRESQL RUNBOOK

4.1 Pre-Migration Assessment & Planning

4.1.1 Source Database Discovery

MySQL/MariaDB Source Discovery

```
-- Database Size
SELECT table_schema AS db,
       ROUND(SUM(data_length+index_length)/1024/1024/1024, 2) AS size_gb
FROM information_schema.tables
WHERE table_schema = 'your_database' GROUP BY table_schema;

-- Object Inventory
SELECT 'TABLE' AS type, COUNT(*) FROM information_schema.tables
WHERE table_schema='your_database' AND table_type='BASE TABLE'
UNION ALL SELECT 'VIEW', COUNT(*) FROM information_schema.views
WHERE table_schema='your_database'
UNION ALL SELECT 'ROUTINE', COUNT(*) FROM information_schema.routines
WHERE routine_schema='your_database'
UNION ALL SELECT 'TRIGGER', COUNT(*) FROM information_schema.triggers
WHERE trigger_schema='your_database';

-- Auto-increment columns (need SERIAL/IDENTITY mapping)
SELECT table_name, column_name, column_type, extra
FROM information_schema.columns WHERE table_schema='your_database'
AND extra LIKE '%auto_increment%';

-- ENUM/SET columns (PG conversion needed)
SELECT table_name, column_name, column_type
FROM information_schema.columns WHERE table_schema='your_database'
AND data_type IN ('enum', 'set');

-- FULLTEXT Indexes (need tsvector/GIN mapping)
SELECT DISTINCT table_name, index_name
FROM information_schema.statistics WHERE table_schema='your_database'
AND index_type = 'FULLTEXT';
```

4.2 AWS SCT -- Schema Conversion for MySQL/MariaDB

MySQL/MariaDB Feature	PostgreSQL Equivalent	Notes
AUTO_INCREMENT	SERIAL / IDENTITY	SERIAL = implicit sequence
ENUM('a','b','c')	CREATE TYPE AS ENUM	Or VARCHAR + CHECK constraint
SET('a','b','c')	TEXT[] or JSONB	No SET type in PG
TINYINT(1) (boolean)	BOOLEAN	Map TINYINT(1) -> BOOLEAN
UNSIGNED integers	CHECK or larger type	PG has no UNSIGNED
DATETIME	TIMESTAMP	Direct mapping
ON UPDATE CURRENT_TIMESTAMP	Trigger function	Create auto-update trigger
FULLTEXT INDEX	GIN + tsvector	to_tsvector() and GIN index

MySQL/MariaDB Feature	PostgreSQL Equivalent	Notes
IF() function	CASE WHEN ... END	Rewrite IF(cond,a,b)
IFNULL()	COALESCE()	Direct replacement
GROUP_CONCAT()	STRING_AGG()	STRING_AGG(col,',')
LIMIT n, m	LIMIT m OFFSET n	Argument order swapped!
REPLACE INTO	INSERT ON CONFLICT DO UPDATE	PG upsert syntax
INSERT IGNORE	INSERT ON CONFLICT DO NOTHING	PG conflict handling
ENGINE=InnoDB	Remove	PG single storage engine
TINYTEXT/MEDIUMTEXT/LONGTEXT	TEXT	All -> PG TEXT
TINYBLOB/.../LONGBLOB	BYTEA	All -> PG BYTEA
JSON (MySQL 5.7+)	JSONB	JSONB is indexable in PG
Backtick quoting `col`	Double-quote or lowercase	Avoid quoting if possible

4.3 MySQL/MariaDB Source Configuration for DMS

MySQL/MariaDB Source Configuration (Combined)

```
# In my.cnf / my.cnf.d/server.cnf:
[mysqld]
server-id            = 1
log_bin              = /var/log/mysql/mysql-bin
binlog_format        = ROW          # MUST be ROW for DMS
binlog_row_image     = FULL        # Required for DMS CDC
expire_logs_days     = 7
max_binlog_size      = 256M
binlog_checksum      = NONE        # DMS requires NONE

# MariaDB 10.2+ additionally:
# binlog_annotate_row_events = OFF

# Restart after changes:
# sudo systemctl restart mysqld      (MySQL)
# sudo systemctl restart mariadb     (MariaDB)

# Verify configuration
SHOW VARIABLES LIKE 'log_bin';        -- ON
SHOW VARIABLES LIKE 'binlog_format';  -- ROW
SHOW VARIABLES LIKE 'binlog_row_image'; -- FULL
SHOW VARIABLES LIKE 'binlog_checksum'; -- NONE

# Create DMS User
CREATE USER 'dms_user'@'%' IDENTIFIED BY 'ComplexP@ss2024!';
GRANT SELECT ON your_database.* TO 'dms_user'@'%';
GRANT REPLICATION CLIENT ON *.* TO 'dms_user'@'%';
GRANT REPLICATION SLAVE ON *.* TO 'dms_user'@'%';
GRANT PROCESS, SHOW DATABASES ON *.* TO 'dms_user'@'%';
FLUSH PRIVILEGES;

# For RDS MySQL:
# CALL mysql.rds_set_configuration('binlog retention hours', 168);
```

WARNING

MariaDB 10.5+: REPLICATION SLAVE was renamed to REPLICATION REPLICA. Use: GRANT REPLICATION REPLICA ON *.* TO 'dms_user'@'%';

4.4 DMS Endpoints & Migration Task

MySQL/MariaDB DMS Endpoint

```
# MySQL/MariaDB Source Endpoint
aws dms create-endpoint \
  --endpoint-identifier mysql-source-prod \
  --endpoint-type source \
  --engine-name mysql \
  --server-name mysql-prod.company.internal \
  --port 3306 --database-name your_database \
  --username dms_user --password 'ComplexP@ss2024!' \
  --extra-connection-attributes \
    'initstmt=SET FOREIGN_KEY_CHECKS=0;parallelLoadThreads=4'

# For MariaDB: --engine-name mariadb
# Target endpoint: same as Section 2.4.3
```

4.5 Alternative: pgloader (Single Command Migration)

For MySQL-to-PG migrations without CDC, pgloader offers the fastest path:

pgloader Quick Start

```
# Install pgloader
sudo dnf install -y pgloader # AlmaLinux/RHEL

# Simple one-liner migration:
pgloader mysql://dms_user:pass@mysql-prod:3306/your_database \
  postgresql://pg_admin:pass@pg-target:5432/your_database

# With configuration file for more control:
pgloader mysql-to-pg.load
```

pgloader Configuration File

```
-- pgloader Configuration File: mysql-to-pg.load
LOAD DATABASE
  FROM mysql://dms_user:pass@mysql-prod:3306/your_database
  INTO postgresql://pg_admin:pass@pg-target:5432/your_database

WITH include drop, create tables, create indexes,
     reset sequences, workers = 8, concurrency = 4,
     batch rows = 25000, prefetch rows = 100000

SET PostgreSQL PARAMETERS
  maintenance_work_mem to '512MB',
  work_mem to '64MB'

CAST type tinyint to smallint using byte-to-smallint,
     type int with extra auto_increment
     to serial when (= default "auto_increment"),
     type tinyint when (= precision 1)
     to boolean using tinyint-to-boolean

ALTER SCHEMA 'your_database' RENAME TO 'public'

BEFORE LOAD DO
  $$ CREATE EXTENSION IF NOT EXISTS "uuid-oss"; $$;
```

TIP

pgloader vs DMS: pgloader = best for one-time migration, no CDC, <500 GB, single command. DMS = best for large databases, CDC requirement, near-zero downtime, parallel load + validation.

4.6 Data Type Mapping -- MySQL/MariaDB to PostgreSQL

MySQL/MariaDB Type	PostgreSQL Type	Notes
TINYINT	SMALLINT	PG has no TINYINT
TINYINT(1)	BOOLEAN	Commonly used as boolean
MEDIUMINT	INTEGER	PG has no MEDIUMINT
INT / BIGINT	INTEGER / BIGINT	Direct mapping
FLOAT / DOUBLE	REAL / DOUBLE PRECISION	Direct mapping
DECIMAL(p,s)	NUMERIC(p,s)	Direct mapping
CHAR(n) / VARCHAR(n)	CHAR(n) / VARCHAR(n)	Direct mapping
TEXT / MEDIUMTEXT / LONGTEXT	TEXT	All -> PG TEXT (unlimited)
BINARY / VARBINARY / BLOB	BYTEA	All -> BYTEA
DATE / TIME	DATE / TIME	Direct mapping
DATETIME	TIMESTAMP	Direct mapping (no TZ)
TIMESTAMP	TIMESTAMPTZ	MySQL TIMESTAMP is UTC-aware
YEAR	SMALLINT	No YEAR type in PG

MySQL/MariaDB Type	PostgreSQL Type	Notes
ENUM('a','b')	CREATE TYPE AS ENUM	Or VARCHAR + CHECK
SET('a','b')	TEXT[]	Array or JSONB
JSON	JSONB	JSONB preferred (indexable)
BIT(n)	BIT(n)	Direct mapping
GEOMETRY / POINT / etc.	PostGIS geometry	Requires PostGIS

4.7 MySQL Dialect to PostgreSQL Conversion

MySQL -- Before

```

-----
-- MYSQL (Before)
-----
DELIMITER //
CREATE PROCEDURE sp_process_orders(IN p_date DATE)
BEGIN
    DECLARE v_count INT DEFAULT 0;
    DECLARE v_total DECIMAL(10,2) DEFAULT 0.00;
    DECLARE done INT DEFAULT FALSE;
    DECLARE cur CURSOR FOR
        SELECT amount FROM orders WHERE order_date = p_date;
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;
    OPEN cur;
    read_loop: LOOP
        FETCH cur INTO v_total;
        IF done THEN LEAVE read_loop; END IF;
        SET v_count = v_count + 1;
    END LOOP;
    CLOSE cur;
    INSERT INTO order_summary(summary_date, total_orders, total_amount)
    VALUES(p_date, v_count, v_total)
    ON DUPLICATE KEY UPDATE
        total_orders = v_count, total_amount = v_total;
END //
DELIMITER ;

```

```

-----
-- PostgreSQL PL/pgSQL (After)
-----
CREATE OR REPLACE PROCEDURE sp_process_orders(p_date DATE)
LANGUAGE plpgsql AS $$
DECLARE
    v_count  INT := 0;
    v_total  NUMERIC(10,2) := 0.00;
    v_amount NUMERIC(10,2);
    cur CURSOR FOR
        SELECT amount FROM orders WHERE order_date = p_date;
BEGIN
    OPEN cur;
    LOOP
        FETCH cur INTO v_amount;
        EXIT WHEN NOT FOUND;
        v_total := v_total + v_amount;
        v_count := v_count + 1;
    END LOOP;
    CLOSE cur;
    INSERT INTO order_summary(summary_date,total_orders,total_amount)
    VALUES(p_date, v_count, v_total)
    ON CONFLICT (summary_date) DO UPDATE SET
        total_orders = EXCLUDED.total_orders,
        total_amount = EXCLUDED.total_amount;
END; $$;

```

Key MySQL -> PG Conversion Rules:

- Remove DELIMITER // ... //; -- PG uses \$\$ dollar quoting
- Replace DECLARE CONTINUE HANDLER with EXIT WHEN NOT FOUND
- Replace ON DUPLICATE KEY UPDATE with ON CONFLICT DO UPDATE SET
- Replace GROUP_CONCAT() with STRING_AGG()
- Replace LIMIT offset,count with LIMIT count OFFSET offset
- Replace AUTO_INCREMENT with GENERATED ALWAYS AS IDENTITY or SERIAL
- Replace backtick quoting with double-quote or lowercase identifiers

4.8 Post-Migration Validation & Cutover

Follow Section 2.8 pattern. MySQL-specific additions:

- Migrate MySQL Events -> pg_cron or pg_timetable
- Convert ENUM types: create PG enum types, then ALTER TABLE columns
- Rebuild FULLTEXT indexes as GIN indexes with tsvector columns
- Update ORM settings (e.g., Django: ENGINE='django.db.backends.postgresql')
- Replace MySQL JDBC driver with PostgreSQL JDBC (org.postgresql.Driver)

PART V -- CROSS-CUTTING CONCERNS

5.1 AWS IAM Roles & Security Setup

DMS requires IAM roles with specific permissions for accessing source/target databases, S3 (for LOB storage), CloudWatch (for logging), and KMS (for encryption).

IAM Roles for DMS

```
# Create DMS VPC Role (required for VPC access)
aws iam create-role \
  --role-name dms-vpc-role \
  --assume-role-policy-document '{
    "Version": "2012-10-17",
    "Statement": [{
      "Effect": "Allow",
      "Principal": {"Service": "dms.amazonaws.com"},
      "Action": "sts:AssumeRole"
    }]
  }'

aws iam attach-role-policy \
  --role-name dms-vpc-role \
  --policy-arn arn:aws:iam::aws:policy/service-role/AmazonDMSVPCManagementRole

# Create DMS CloudWatch Role (for task logging)
aws iam create-role \
  --role-name dms-cloudwatch-logs-role \
  --assume-role-policy-document '{
    "Version": "2012-10-17",
    "Statement": [{
      "Effect": "Allow",
      "Principal": {"Service": "dms.amazonaws.com"},
      "Action": "sts:AssumeRole"
    }]
  }'

aws iam attach-role-policy \
  --role-name dms-cloudwatch-logs-role \
  --policy-arn arn:aws:iam::aws:policy/service-role/AmazonDMSCloudWatchLogsRole

# For KMS encryption at rest (optional but recommended):
# Create KMS key and grant dms.amazonaws.com access
```

5.2 Network Configuration (VPN / Direct Connect)

For on-premises source databases, DMS requires network connectivity via VPN or Direct Connect.

Network Configuration

```
# DMS Network Architecture for On-Premises Sources:
#
# [On-Prem Oracle/MSSQL/MySQL]
#   |
# [Site-to-Site VPN OR AWS Direct Connect]
#   |
# [VPC - Private Subnet]
#   |
# [DMS Replication Instance] <--> [Target RDS/Aurora PG]
#
# Key requirements:
# 1. DMS RI in private subnet with route to on-prem via VPN/DX
# 2. Security Groups:
#   - DMS RI SG: outbound to source DB port (1521/1433/3306)
#   - DMS RI SG: outbound to target DB port (5432)
#   - Target DB SG: inbound from DMS RI SG on port 5432
# 3. On-prem firewall: allow DMS RI IP range on source DB port
# 4. Replication subnet group spans >= 2 AZs

# Create Replication Subnet Group
aws dms create-replication-subnet-group \
  --replication-subnet-group-identifier dms-subnet-group \
  --replication-subnet-group-description 'DMS private subnets' \
  --subnet-ids subnet-aaa111 subnet-bbb222
```

5.3 DMS CloudWatch Monitoring & Alerting

Set up CloudWatch alarms for critical DMS metrics:

Metric	Alarm Threshold	Action
CDCLatencySource	> 300 seconds (5 min)	Investigate source log reader performance
CDCLatencyTarget	> 300 seconds (5 min)	Investigate target apply bottleneck
CDCIncomingChanges	Rising trend with no apply	Check for target blocking / locks
FreeStorageSpace	< 20% of allocated	Increase RI storage immediately
CPUUtilization	> 80% sustained	Upgrade to larger RI instance class
FreeableMemory	< 500 MB	Reduce MaxFullLoadSubTasks or upgrade
SwapUsage	> 100 MB	Upgrade instance class; memory pressure
NetworkTransmitThroughput	Near instance limit	Check for network saturation

CloudWatch Alarm Example

```
# Create CloudWatch Alarm for CDC Latency
aws cloudwatch put-metric-alarm \
  --alarm-name dms-cdc-latency-high \
  --alarm-description 'DMS CDC lag exceeds 5 minutes' \
  --metric-name CDCLatencyTarget \
  --namespace AWS/DMS \
  --dimensions Name=ReplicationInstanceIdentifier,Value=oracle-to-pg-ri \
  --statistic Average --period 300 --threshold 300 \
  --comparison-operator GreaterThanThreshold \
  --evaluation-periods 2 \
  --alarm-actions arn:aws:sns:us-east-1:123456:dba-alerts
```

5.4 LOB Migration Strategies

Large Object (LOB) columns (BLOB, CLOB, NCLOB, IMAGE, etc.) are the most common source of DMS performance issues. Choose the right LOB mode:

LOB Mode	Setting	Max Size	Performance	Use When
Limited LOB	LimitedSizeLobMode=true LobMaxSize=32768	32 KB (configurable)	Fast (best perf)	Most LOBs < 32 KB; truncation acceptable
Full LOB	FullLobMode=true	Unlimited	Slow (row-by-row)	Any LOB > LobMaxSize; no truncation allowed
Inline LOB	InlineLobMaxSize=32768 LobMaxSize=102400	102 KB (two-tier)	Medium (best balance)	Mixed sizes; small LOBs inlined, large separate

TIP

Production Recommendation: Start with Inline LOB mode (InlineLobMaxSize=32768, LobMaxSize=102400). This handles 95%+ of LOBs inline (fast) while falling back to full LOB mode for larger values. Monitor DMS logs for LOB truncation warnings.

5.5 Large Table Parallel Load Strategy

For tables with 100M+ rows, DMS supports parallel load by partitioning the table into ranges. This dramatically improves full load performance.

Parallel Load for Large Tables

```
# In table-mappings.json, add parallel-load settings:
{
  "rules": [
    {
      "rule-type": "selection",
      "rule-id": "1",
      "rule-name": "large-orders-table",
      "object-locator": {
        "schema-name": "YOUR_SCHEMA",
        "table-name": "ORDERS"
      },
      "rule-action": "include",
      "parallel-load": {
        "type": "ranges",
        "columns": ["ORDER_ID"],
        "boundaries": [
          ["10000000"],
          ["20000000"],
          ["30000000"],
          ["40000000"]
        ]
      }
    }
  ]
}

# This splits the table into 5 segments:
#   Segment 1: ORDER_ID < 10000000
#   Segment 2: 10000000 <= ORDER_ID < 20000000
#   ...etc. Each loaded in parallel by DMS.
```

5.6 PostgreSQL Target Performance Tuning

Tune the target PostgreSQL instance BEFORE starting migration:

PostgreSQL Target Tuning (Pre-Migration)

```
-- Apply these settings on the TARGET PostgreSQL BEFORE migration
-- (Revert after migration completes)

-- Disable synchronous commit for faster writes during load
ALTER SYSTEM SET synchronous_commit = 'off';

-- Increase checkpoint distance to reduce I/O during bulk load
ALTER SYSTEM SET max_wal_size = '16GB';
ALTER SYSTEM SET checkpoint_completion_target = 0.9;

-- Increase maintenance work memory for index creation
ALTER SYSTEM SET maintenance_work_mem = '2GB';

-- Disable autovacuum during full load (re-enable after!)
ALTER SYSTEM SET autovacuum = off;

-- Increase WAL writer delay
ALTER SYSTEM SET wal_writer_delay = '500ms';

-- Reload configuration
SELECT pg_reload_conf();

-- AFTER migration completes, REVERT critical settings:
ALTER SYSTEM SET synchronous_commit = 'on';
ALTER SYSTEM SET autovacuum = on;
ALTER SYSTEM RESET max_wal_size;
ALTER SYSTEM RESET maintenance_work_mem;
SELECT pg_reload_conf();

-- Run full ANALYZE after migration
ANALYZE VERBOSE;
```

CRITICAL

CRITICAL: Re-enable autovacuum and synchronous_commit IMMEDIATELY after migration completes. Running production workload with autovacuum=off will cause table bloat and performance degradation within days.

5.7 Terraform IaC for DMS Infrastructure

Infrastructure as Code (IaC) ensures reproducible DMS deployments. Below is a production Terraform module for the complete DMS stack:

Terraform DMS Module

```
# main.tf -- DMS Infrastructure Module

resource "aws_dms_replication_subnet_group" "dms" {
  replication_subnet_group_id = "dms-subnet-group"
  replication_subnet_group_description = "DMS private subnets"
  subnet_ids = var.private_subnet_ids
}

resource "aws_dms_replication_instance" "main" {
  replication_instance_id = "${var.project}-ri"
  replication_instance_class = var.dms_instance_class # dms.c5.2xlarge
  allocated_storage = var.dms_storage # 200
  multi_az = true
  publicly_accessible = false
  vpc_security_group_ids = [aws_security_group.dms.id]
  replication_subnet_group_id = aws_dms_replication_subnet_group.dms.id
  engine_version = "3.5.3"
  tags = merge(var.tags, { Name = "${var.project}-dms-ri" })
}

resource "aws_dms_endpoint" "source" {
  endpoint_id = "${var.project}-source"
  endpoint_type = "source"
  engine_name = var.source_engine # oracle / sqlserver / mysql
  server_name = var.source_host
  port = var.source_port
  database_name = var.source_db
  username = var.source_user
  password = var.source_password
  extra_connection_attributes = var.source_extra_attrs
  tags = var.tags
}

resource "aws_dms_endpoint" "target" {
  endpoint_id = "${var.project}-target"
  endpoint_type = "target"
  engine_name = "aurora-postgresql"
  server_name = var.target_host
  port = 5432
  database_name = var.target_db
  username = var.target_user
  password = var.target_password
  tags = var.tags
}

resource "aws_dms_replication_task" "main" {
  replication_task_id = "${var.project}-task"
  replication_instance_arn = aws_dms_replication_instance.main.replication_instance_arn
  source_endpoint_arn = aws_dms_endpoint.source.endpoint_arn
  target_endpoint_arn = aws_dms_endpoint.target.endpoint_arn
  migration_type = "full-load-and-cdc"
  table_mappings = file("${path.module}/table-mappings.json")
  replication_task_settings = file("${path.module}/task-settings.json")
  tags = var.tags
}
```

TIP

Terraform Best Practice: Store source/target passwords in AWS Secrets Manager and reference via `data.aws_secretsmanager_secret_version`. Never hardcode credentials in Terraform state files.

PART VI -- APPENDICES

A. AWS DMS Instance Sizing Guide

Source Size	Tables	Recommended Instance	Storage	Notes
< 100 GB	< 50	dms.c5.large (2 vCPU, 4 GB)	50 GB	Dev/test
100-500 GB	50-200	dms.c5.xlarge (4 vCPU, 8 GB)	100 GB	Small prod
500 GB-2 TB	200-500	dms.c5.2xlarge (8 vCPU, 16 GB)	200 GB	Std. prod
2-5 TB	500-1K	dms.c5.4xlarge (16 vCPU, 32 GB)	500 GB	Large prod
> 5 TB	> 1K	dms.c5.9xlarge+ (36 vCPU, 72 GB)	1 TB+	Enterprise

B. Common DMS Error Codes & Resolution

Error	Cause	Resolution
FATAL_ERROR: Sorter	RI storage full	Increase storage; reduce MaxFullLoadSubTasks
LOB column has NULL	U+0000 in text column	DMS converts to space; verify app compat
Cannot connect to source	SG / network ACL	Verify SG allows DMS RI -> Source on DB port
CDC task stopped	Binlog/archivelog expired	Increase retention; restart with fresh full load
Duplicate key violation	FK enabled during load	Set CreatePkAfterFullLoad=true; disable FKs
Data validation mismatch	TZ/encoding differences	Check session_replication_role and TZ settings
Table not found (target)	Case sensitivity	Add convert-lowercase transformation rule
Slow full load	Too many parallel tasks	Reduce MaxFullLoadSubTasks; upgrade instance
Memory pressure	Large LOB columns	Use LimitedSizeLobMode; set LobMaxSize
Replication slot missing	PG slot dropped	Recreate task or restart fresh

C. Rollback Strategy Template

CRITICAL

Rollback Decision Matrix:

- Data validation passes AND smoke tests pass -> Proceed with cutover
- Data validation fails (mismatch > 0.01%) -> Rollback immediately
- Smoke tests fail with data errors -> Rollback to source
- Smoke tests fail with code errors only -> Fix code, re-test (no rollback)
- Maximum rollback window: 4 hours from cutover start
- After 4 hours: evaluate fix-forward vs. rollback

Rollback Procedure

```
# ROLLBACK PROCEDURE
# Step 1: Stop all application traffic to PostgreSQL target
# Step 2: DMS task was stopped; source DB is preserved
# Step 3: Revert application connection strings to source
kubectl set env deployment/myapp DB_HOST=oracle-prod.company.internal
kubectl rollout restart deployment/myapp
# Step 4: Verify source DB is operational
# Step 5: Resume application writes to source
# Step 6: Document failure reason for post-mortem
# Step 7: Plan remediation before next cutover attempt
```

D. Migration Checklist (Printable)

Phase	Task	Done
ASSESS	Source DB discovery (size, objects, PL/SQL complexity)	☐
ASSESS	Run Ora2Pg assessment (Oracle) or SCT report	☐
ASSESS	Identify unsupported features & manual rewrite items	☐
ASSESS	Estimate timeline and resource allocation	☐
SCHEMA	Install AWS SCT with JDBC drivers	☐
SCHEMA	Convert DDL (tables, indexes, constraints)	☐
SCHEMA	Convert views and materialized views	☐
SCHEMA	Convert functions/procedures/packages -> PL/pgSQL	☐
SCHEMA	Convert triggers	☐
SCHEMA	Create PG extensions (uuid-osp, pgcrypto, etc.)	☐
SCHEMA	Apply schema to target (WITHOUT foreign keys)	☐
INFRA	Create IAM roles (VPC, CloudWatch, KMS)	☐
INFRA	Configure VPN/DirectConnect network path	☐
INFRA	Create replication subnet group	☐
INFRA	Tune target PG parameters for bulk load	☐
DATA	Configure source DB (logging, grants, user)	☐
DATA	Create DMS replication instance (C5, Multi-AZ)	☐
DATA	Create & test source endpoint	☐
DATA	Create & test target endpoint	☐
DATA	Create task (Full Load + CDC) with lowercase rules	☐
DATA	Start task and monitor full load	☐
DATA	Verify full load completion (row counts)	☐
DATA	Monitor CDC replication lag	☐

Phase	Task	Done
DATA	Set up CloudWatch alarms for DMS metrics	☐
VALID	Row count comparison (all tables)	☐
VALID	Checksum validation (critical tables)	☐
VALID	Application integration testing	☐
VALID	Performance benchmarking on target	☐
CUT	Schedule maintenance window	☐
CUT	Stop application writes	☐
CUT	Wait for CDC lag = 0	☐
CUT	Stop DMS task	☐
CUT	Reset sequences to MAX(id) + 1	☐
CUT	Apply deferred indexes and foreign keys	☐
CUT	Revert PG tuning params (autovacuum, sync_commit)	☐
CUT	Run ANALYZE on all tables	☐
CUT	Switch application connection strings	☐
CUT	Smoke test critical workflows	☐
CUT	Monitor error logs for 30 minutes	☐
CUT	Declare SUCCESS or ROLLBACK	☐
POST	Schedule source DB decommission (30-day hold)	☐
POST	Document lessons learned	☐
POST	Update runbooks & monitoring for PostgreSQL	☐

E. Cost Estimation Reference

AWS DMS pricing is based on replication instance hours. Below are approximate costs (us-east-1, on-demand, subject to change):

Instance Class	vCPU	Memory	Approx. \$/hr	Monthly (24x7)
dms.c5.large	2	4 GB	\$0.154	~\$111
dms.c5.xlarge	4	8 GB	\$0.308	~\$222
dms.c5.2xlarge	8	16 GB	\$0.616	~\$443
dms.c5.4xlarge	16	32 GB	\$1.232	~\$887
dms.c5.9xlarge	36	72 GB	\$2.772	~\$1,996

TIP

Cost Optimization: DMS charges only while the replication instance is running. For one-time full-load-only migrations, stop the instance immediately after completion. For CDC, the instance must run continuously until cutover. Multi-AZ doubles the cost but is recommended for production migrations to avoid restart delays.